Python Exam Paper

Q1. Answer the following questions (Any Six). [6 × 2 = 12 M]

(Each part here is worth 2 marks — concise, precise answers with examples where helpful.)

a) Enlist different attributes of a Python object.

- id(obj) — unique identifier (memory address-like).
- type(obj) — the object's class/type.
- `__dict__`: A dictionary containing the object's writable attributes.
- `__doc__`: The docstring of the object, if defined.
- `__class__`: The class to which the object belongs.
- dir(obj) — list of attributes/methods.
- __class__ — reference to the class object.
  Example:

```
s = "hello"

print(id(s))

print(type(s))      # <class 'str'>

print(s.__class__)   # <class 'str'>

print(dir(s))       # methods and attributes
```

b) Out[6]: array(['i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]]) — Write Python code to generate above output.
Interpretation: produce an array-like representation where first element 'i' and second is list 0–9. Example using numpy.array (it will make object dtype):

```
import numpy as np

arr = np.array(['i', list(range(10))], dtype=object)

print(arr)  # array(['i', list([0,1,...,9])], dtype=object)
```

c) What are Python's scalar data types?
Scalar data types represent single, atomic values. The main built-in scalar types in Python are:

int, float, complex, bool, str (often considered scalar for single value), and bytes (binary sequences).
Example: x = 3 (int), y = 3.14 (float), z = True (bool).

d) Write a Python code to get the following output: {1:1, 3:9, 5:25, 7:49, 9:81}
Use dictionary comprehension:

```
d = {i: i*i for i in range(1, 10, 2)}

print(d)  # {1:1, 3:9, 5:25, 7:49, 9:81}
```

e) Given code:

```
a = 'hello world'

b = a.capitalize()
```

```
print('Original String:', a)

print('New String:', b)
```

What will be the output of above code?
Output:

Original String: hello world

New String: Hello world

Explanation: capitalize() returns a new string with first character uppercase and the rest lowercased; original a unchanged.

f) How does Python's is operator work?
is checks identity: whether two references point to the same object (same memory/identity), i.e., id(a) == id(b). Example:

```
a = [1,2]

b = a

c = [1,2]

print(a is b)  # True

print(a is c)  # False
```

g) Explain what the type() function does in Python.
type(obj) returns the type (class) of obj. When called with three arguments type(name, bases, dict) it dynamically creates a new class. Example:

```
print(type(3))       # <class 'int'>

print(type("hi"))    # <class 'str'>

MyClass = type('MyClass', (), {'x': 5})  # creates class MyClass dynamically
```

Q2. **a) What is a for loop in Python, and how is it different from a while loop?**
A for loop iterates directly over items of a sequence/iterable (list, tuple, string, range, generator). It repeats body for each element. A while loop repeats as long as a condition is True — condition-controlled.
Example for:

```
for i in range(3):
    print(i)
```

Example while:

```
i = 0

while i < 3:
    print(i)
    i += 1
```

Difference: for is typically used when number/items of iterations known; while when looping condition depends on runtime or external events.

b) How does a Python interpreter work, and what is its role?
The Python interpreter is a virtual machine that executes Python code. Its role is to translate and run high-level Python code into instructions the computer can understand.

How it works:

1. **Source Code:** The programmer writes code in a .py file.
2. **Compilation (to Bytecode):** The interpreter first compiles the source code into a simpler, intermediate language called **bytecode** (stored in .pyc files). This step checks for syntax errors.
3. **Execution (by PVM):** The compiled bytecode is then executed by the **Python Virtual Machine (PVM)**. The PVM is the runtime engine of Python that reads the bytecode instructions and executes them line by line.

c) What is the purpose of the __init__.py file in a Python package?
The __init__.py file serves two primary purposes:

1. **Directory Marker:** It signals to the Python interpreter that the directory should be treated as a **package**, allowing for the import of modules from that directory.
2. **Package Initialization:** It can contain initialization code for the package (e.g., setting up package-level variables, imports, or running setup routines). It is executed when the package is first imported.

Example:
A directory structure mypackage/ containing __init__.py and mymodule.py becomes an importable package.

```
# In another script, you can now import as:

import mypackage.mymodule

# or

from mypackage import mymodule
```

d) What is the purpose of the try-except block in Python?
To handle exceptions (errors) gracefully: run code in try, catch exceptions in except to prevent program crash, optionally use else (on no exception) and finally (always run cleanup). Example:

```
try:
    x = 1/0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Cleanup")
```

e) Explain the concept of list slicing in Python.

Slicing extracts subsequences using start:stop:step. It returns a new list/string/sequence. seq[a:b] includes indices a to b-1. Examples:

```
lst = [0,1,2,3,4,5]

print(lst[1:4])   # [1,2,3]

print(lst[:3])    # [0,1,2]

print(lst[::2])   # [0,2,4]

print(lst[-3:])   # last 3 elements
```

Q3. **a) How do you define a function in Python? Provide an example.**

Use def followed by name, parameters, optional docstring, and return. Example:

```
def factorial(n):
    """Return factorial of n (n!)."""
    if n < 0:
        raise ValueError("n must be non-negative")
    result = 1
    for i in range(2, n+1):
        result *= i
    return result


print(factorial(5))  # 120
```

Also can use lambda for short anonymous functions: f = lambda x: x*x.

b) Explain the difference between continue and break statements in loops.

- break exits the nearest enclosing loop immediately; loop ends and execution continues after loop.
- continue skips the remainder of the loop body for this iteration and goes to next iteration (evaluates loop condition or moves to next item).
  Example:

```
for i in range(5):
    if i == 2:
        continue   # skip printing 2
    if i == 4:
        break      # stop loop at 4
    print(i)
```

Output: 0 1 3

c) What is difference between compiler and interpreter?

- Compiler: Translates entire source code to machine code/binary ahead of execution; produces an executable. Faster execution at runtime but compile step required (e.g., C, C++).
- Interpreter: Reads and executes code line-by-line or compiles to intermediate form executed by a VM; executes dynamically (e.g., Python). Easier debugging and portability; generally slower runtime. Python uses an interpreter that compiles to bytecode then runs the VM (hybrid approach).

Feature	Compiler	Interpreter
Execution	Scans and translates the entire program into machine code at once.	Translates and executes the program line by line.
Speed	Generally faster for execution as the translation is done beforehand.	Slower for execution as it translates during runtime.
Error Output	Displays all errors after compiling the entire program.	Stops at the first error it encounters and displays it immediately.
Portability	Generates platform-specific machine code.	More portable; the same bytecode can run on any platform with an interpreter.
Examples	C, C++, Java (to bytecode)	Python, JavaScript, Ruby

d) What is a set in Python?

A set is an unordered, mutable collection of unique, immutable objects. It is defined by values separated by commas inside curly braces {}.

Key Properties:

- **Unordered:** Items have no defined order.
- **Mutable:** You can add or remove items.
- **Unique:** Duplicate elements are automatically removed.
- **Elements must be hashable:** Typically, this means they must be immutable (e.g., numbers, strings, tuples).

Example:

```
my_set = {1, 2, 3, 2, 1} # Duplicates are removed

print(my_set) # Output: {1, 2, 3}


# Common methods

my_set.add(4)   # Adds an element

my_set.remove(2) # Removes an element
```

```
print(my_set) # Output: {1, 3, 4}
```

Q4. Answer the following questions (Any Two). [2 × 4 = 8 M]

(Each ~4 marks — deeper explanation, examples.)

a) What is mutable? Is tuple mutable? Demonstrate any two methods of tuple with suitable example.

- *Mutable* objects can change their content after creation (e.g., list, dict, set). *Immutable* objects cannot be changed in-place (e.g., int, str, tuple, frozenset).
- Tuple is immutable: once created, items cannot be added, removed, or changed in-place. However, if a tuple contains a mutable object (like a list), that inner object can be changed.
  Example showing immutability and two tuple operations (indexing and count — not methods that modify because tuples are immutable):

```
t = (1, 2, 3, 2)

# 1) indexing

print(t[0])       # 1

# 2) count and index (methods available)

print(t.count(2))  # 2   (number of occurrences)

print(t.index(3))  # 2   (first index of value)

# Attempt to modify raises error:

# t[0] = 10  # TypeError: 'tuple' object does not support item assignment
```

If you need changes, convert to list: list(t) modify then tuple() back.

b) Discuss the advantage of Python.
Key advantages:

- Easy to read & write: clean syntax, fewer lines of code.
- Large standard library & ecosystem: math, datetime, collections, os, plus packages (NumPy, pandas, requests, Django).
- Cross-platform & interpreted: runs on many OS without recompiling.
- Rich community & third-party modules: quick to prototype and find libraries for tasks (ML, web, automation).
- Dynamic typing & higher-level data structures: lists, dicts, sets make development fast.
  Example: simple HTTP request using requests package:

```
import requests

r = requests.get('https://api.github.com')

print(r.status_code)
```

c) Explain how the numpy.split() function works and provide its examples.
numpy.split(ary, indices_or_sections, axis=0) splits an array into multiple sub-arrays.

- If indices_or_sections is an integer N, splits into N equal arrays along axis (size must be divisible).
- If it's a list of indices, splits at those index positions. Use np.array_split to allow unequal splits.
  Examples:

```
import numpy as np

a = np.arange(8)

print(np.split(a, 4))          # splits into 4 arrays of size 2

# [array([0,1]), array([2,3]), array([4,5]), array([6,7])]


print(np.split(a, [3,5]))      # splits at index 3 and 5

# [array([0,1,2]), array([3,4]), array([5,6,7])]
```

Q5. **a) How does NumPy handle random number generation, and what are the different types of random functions available?**

NumPy uses the numpy.random module (in modern NumPy, preferred API is numpy.random.default_rng() which returns a Generator using PCG64). Random functions include:

- Generator.integers(low, high, size) — integer randoms.
- Generator.random(size) — floats in [0,1).
- Generator.normal(loc, scale, size) — normal (Gaussian).
- Generator.uniform(low, high, size) — uniform distribution.
- Generator.choice(a, size, replace, p) — sample from array.
- Generator.shuffle(x) — in-place shuffle.
- Generator.permutation(x) — return shuffled copy.
  Example using new API:

```
import numpy as np

rng = np.random.default_rng(seed=42)

print(rng.random(5))           # 5 floats in [0,1)

print(rng.integers(0, 10, size=5))# 5 ints 0-9

print(rng.normal(0, 1, size=3))   # normal samples

print(rng.choice([10,20,30], 3))  # sample choices
```

Importance: default_rng() gives reproducible sequences with seed, and better statistical properties.

b) (This part from image Q5 seems to have more parts (Q5 continued on page 2). I answer the two main Q5 parts — the second likely about Pandas DataFrame and CSV reading — see part (b) and (c) on page 2)

b) What is a DataFrame in Pandas, and how do you create one using a dictionary in Python?
A DataFrame is a 2D tabular labeled data structure with rows and columns (like Excel sheet). You can create from dictionary where keys are column names and values are lists/arrays. Example:

Creating a DataFrame from a dictionary:
You can create a DataFrame by passing a dictionary where the **keys become the column names** and the **values are lists or arrays containing the column data**.

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Cathy'],

    'Age': [25, 30, 22],

    'Salary': [50000, 60000, 52000]}

df = pd.DataFrame(data)

print(df)
```

c) How can you read data from a CSV file into a Pandas DataFrame? Provide a basic example.
Use pd.read_csv() specifying filepath and optional parameters (header, index_col, delimiter, parse_dates). Example:

```
import pandas as pd

df = pd.read_csv('data.csv')          # basic

# Example with options:

df2 = pd.read_csv('data.csv', index_col=0, parse_dates=['date_col'])

print(df.head())
```

Q6. **a) Discuss the NumPy Arrays Manipulation in Python.** ***(Detailed — 6 marks)***

NumPy arrays (numpy.ndarray) are the central data structure in NumPy for numerical computing. Key points:

- Creation: np.array(list), np.arange, np.linspace, np.ones, np.zeros, np.eye (identity).
  Example: a = np.array([[1,2,3],[4,5,6]])
- Shape & reshape: a.shape returns dimensions. Use a.reshape(rows, cols) to change shape (compatible sizes).
- a = np.arange(6) # [0..5]
- b = a.reshape(2,3) # shape (2,3)
- Indexing & slicing: supports multi-dimensional indexing a[i,j], slicing a[:,1], boolean indexing and fancy indexing (using lists/arrays of indices).
- m = np.array([[1,2],[3,4]])
- print(m[0,1]) # 2

- print(m[:,0]) # first column
- Broadcasting: operations between arrays of different shapes where smaller array is virtually expanded to match larger shape following broadcasting rules. Example:
- A = np.ones((3,3))
- b = np.array([1,2,3])
- A + b # adds row-wise using broadcasting
- Vectorized operations: element-wise arithmetic without Python loops; much faster than Python lists.
- x = np.array([1,2,3])
- y = x * 2 # [2,4,6]
- Aggregation & axis ops: sum, mean, max, min, std with axis parameter.
- a.mean(axis=0) # column-wise mean
- Linear algebra & reshaping: dot, matmul, transpose, T, linalg functions.
- np.dot(A, B)
- Concatenation / splitting: np.concatenate, np.vstack, np.hstack, np.split, np.array_split. Example:
- x = np.array([1,2]); y = np.array([3,4])
- np.concatenate([x,y]) # [1,2,3,4]
- Copy vs view: slicing often returns views — modifying view affects original. Use copy() to get independent array.
- Type handling: arrays are homogeneous (single dtype). Use astype() to convert dtype. Example: a.astype(float)

These manipulations make NumPy essential for scientific computing, data analysis, and as the base for libraries such as pandas and TensorFlow.

b) Write a Note on file handling in Python. ***(Detailed — 6 marks)***

File handling in Python involves opening files, reading/writing, and closing them. Key concepts:

- Opening files: open(filename, mode, encoding) — common modes: 'r' (read), 'w' (write, truncates), 'a' (append), 'rb'/'wb' (binary), 'r+' (read/write).
- f = open('example.txt', 'r', encoding='utf-8')
- Reading from files: use read() (whole file), readline() (one line), readlines() (list of lines), or iterate directly:
- with open('example.txt') as f:

- for line in f:
- print(line.strip())

with ensures automatic closing (context manager) even if errors occur.

- Writing to files: write() and writelines():
- with open('out.txt', 'w', encoding='utf-8') as f:
- f.write('Hello\n')
- f.writelines(['line1\n','line2\n'])
- Binary files: use 'rb'/'wb' for images, serialized data (pickle). Example:
- with open('image.png', 'rb') as f:
- data = f.read()
- CSV and structured data: use csv module or pandas for convenient CSV reading/writing (pd.read_csv, df.to_csv). Example using csv:
- import csv
- with open('data.csv', newline='') as csvfile:
- reader = csv.reader(csvfile)
- for row in reader:
- print(row)
- File metadata and os operations: use os and pathlib for checking existence, deleting, renaming:
- from pathlib import Path
- p = Path('example.txt')
- print(p.exists())
- p.rename('newname.txt')
- Best practices: always use with for safety, handle exceptions (e.g., IOError), specify encoding (utf-8) for text files, and close files or rely on context manager.